

CAPITAL FACTORY

Capital Factory can't remember what it built.

An organizational memory for an AI estate that outgrew the people who made it.

Proposal · Sept 2026

This is a success story that ran past its own limits.

2024

Give capable people Claude and let them build.

Year one

It works. Applications appear. Everyone ships.

Today

51 packaged items. Nobody was ever assigned to keep track.

**At five applications,
nobody needs a system
to remember them.**

At fifty-one, the knowledge lives in individual heads and individual repositories, and there is no way to ask the organization what it can already do.

THE PROBLEM

Nobody knows what already exists.

Every figure from Capital Factory's own repository survey.

51

packaged items

31

applications

~10

built more than once

1

evaluation suite

And it is organized by who wrote it

draft-like-jamie

draft-like-drew

jamie-simulator

drew-simulator

meeting-schedule x3

Six of the ten duplicates are duplicated once per person. Nobody did anything wrong: there was no shared place, so copying was the only move the system allowed.

This is already doing damage.

compliance

A safety rule only some apps follow

Four applications write the investor database. Three go through a guardrail. One writes directly.

correctness

A control that quietly diverged

A compliance rule was copied by hand into two repositories. The copies no longer agree.

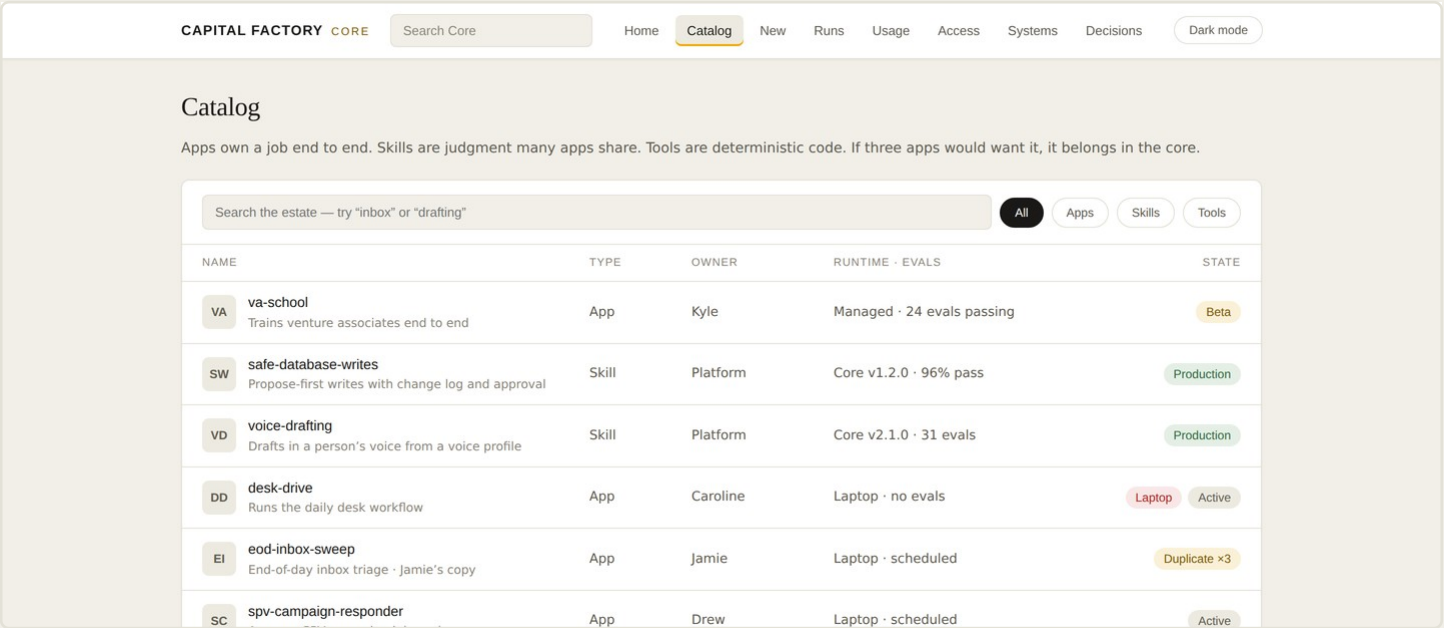
cost

Savings nobody can collect

Swapping a cheaper model into work that does not need a frontier one would have to be found and done separately in every copy.

A compliance control enforced by copy and paste is a control by memory. Memory is the thing that ran out.

So build a catalog.



One place to log in, see everything Capital Factory has built, and launch what you need.

This is what we proposed first.

It is also, as it turns out, what an entire industry sells. We come back to that.

The instinct is right. The shape is wrong, for two reasons.

WHY IT FAILS, ONE

Nobody goes there.

“I don't want our employees to have to worry about what's in the core.”

The person paying for the platform, on whether anyone would use it

They already work somewhere. Four somewheres.

Claude

GitHub

Vercel

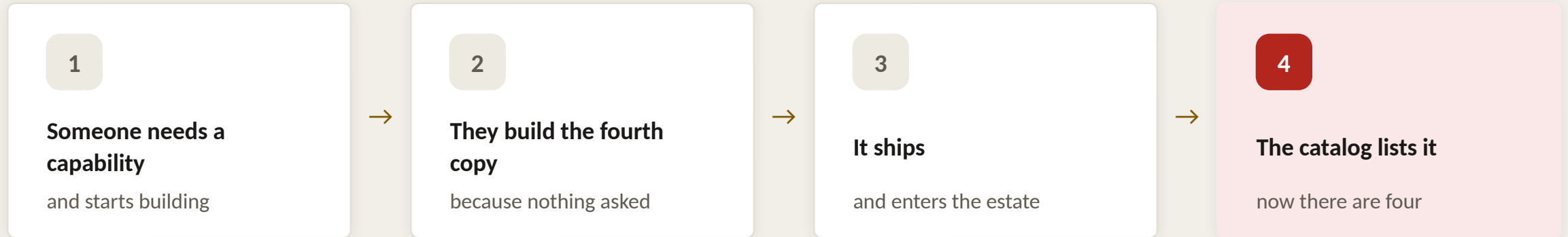
Their own apps

+ a catalog

A fifth destination, maintained part time, competing for attention with the four places the work actually happens. It asks people to stop and visit somewhere before doing the thing they wanted to do.

And even if they did, it finds out too late.

A catalog is a record of what has been built. Follow what that means.



A catalog can tell you that you have a problem. It cannot stop you from creating one.

THE REQUIREMENT

So what would actually work?

1

The answer has to arrive before the work starts.

Not after it ships. Not in a weekly report.

2

Nobody can be required to go and get it.

If using it is a separate step, it becomes optional, and optional means skipped.

3

It has to be useful to whoever is doing the building.

At Capital Factory that is not only a person.

Those three rule out every version of a place people visit. Including a chat interface, which is a catalog with a different input box.

Put the answer where the work already happens.

**Everyone at Capital Factory already builds with Claude.
So don't build a place for people to go. Give Claude access to what the
company knows.**

Nobody visits anything

The question gets asked during planning,
by the tool already open.

Nobody learns a new system

There is nothing to adopt, no login, no
training.

It happens before the work

Which is the only moment duplication can
still be prevented.

Nothing moves. Something gets added underneath.

What executes

stays exactly where it is

- GitHub is GitHub
- Vercel is Vercel
- Applications stay deployed where they are
- No migration, no freeze, nobody stops shipping

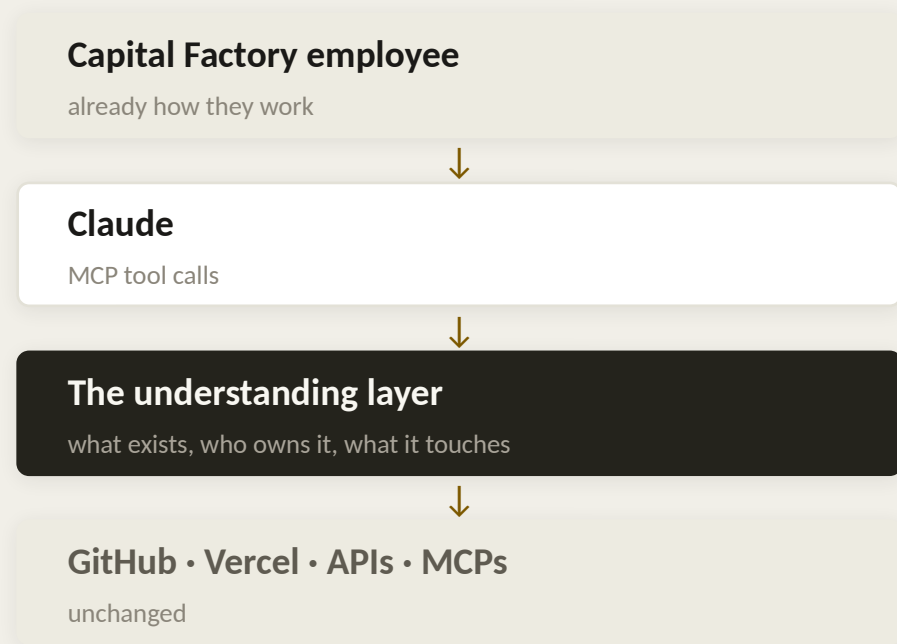
What understands

is the new part

- What have we built
- What can we already do, and where does it live
- Who owns it and what does it touch
- What should a new application follow

The estate does not have an interface problem. It has a memory problem, and memory is what gets added.

Claude already speaks MCP, so nothing gets installed.



Six tools

```
capability_search
capability_get
application_list
standards_get
systems_describe
usage_log
```

Search matches on what a capability does, not what it is called. In an estate organized by person, names are the least reliable signal there is.

It serves descriptions of capabilities, never the data those capabilities touch.

Nick has been here two weeks and will never hear of any of this.

WHAT NICK SEES

NICK

I want an app that takes a startup we're considering and tells me the three best mentor matches.

CLAUDE

What sources for the startup? Who can run it? Does a match need sign-off first?

NICK

Deck and website. Investment team. Yes, I approve.

CLAUDE

Built. It ingests the startup, ranks mentors, and holds each match for you.

WHAT HAPPENED UNDERNEATH

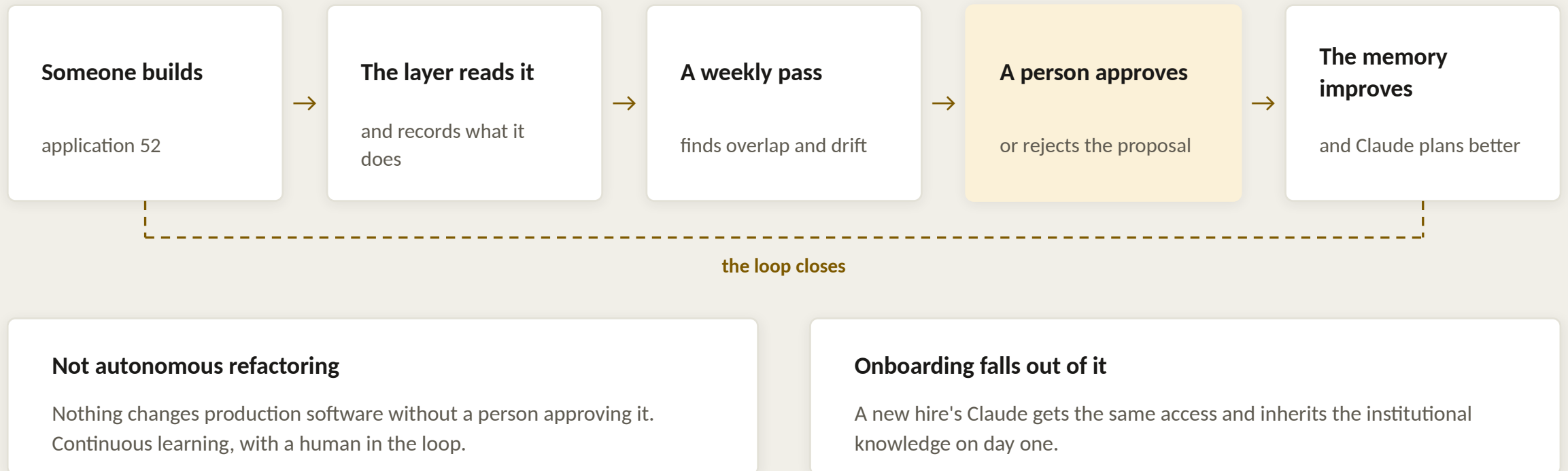
plan decomposed → 7 capabilities

startup-ingest	reused
mentor-rank	reused
approval-gate	reused
email-notify	no match

duplication prevented: 2

The non-match is worth as much as the matches. It is duplication the estate was about to build, visible before it happened.

Every new application makes the next one easier to build.



THE RULE

Not everything duplicated should be merged.

Share it

when a change must reach everyone

The unguarded database write. Fix it once, and every application that touches the investor database inherits the fix. A safety rule only some applications follow is not a rule.

Leave it alone

when a change must not

draft-like-jamie and draft-like-drew stay separate forever. Drew does not want Jamie's improvements arriving in his drafts. Independence is the feature, not the bug.

Four questions decide it, one capability at a time

1 Does a change have to propagate?

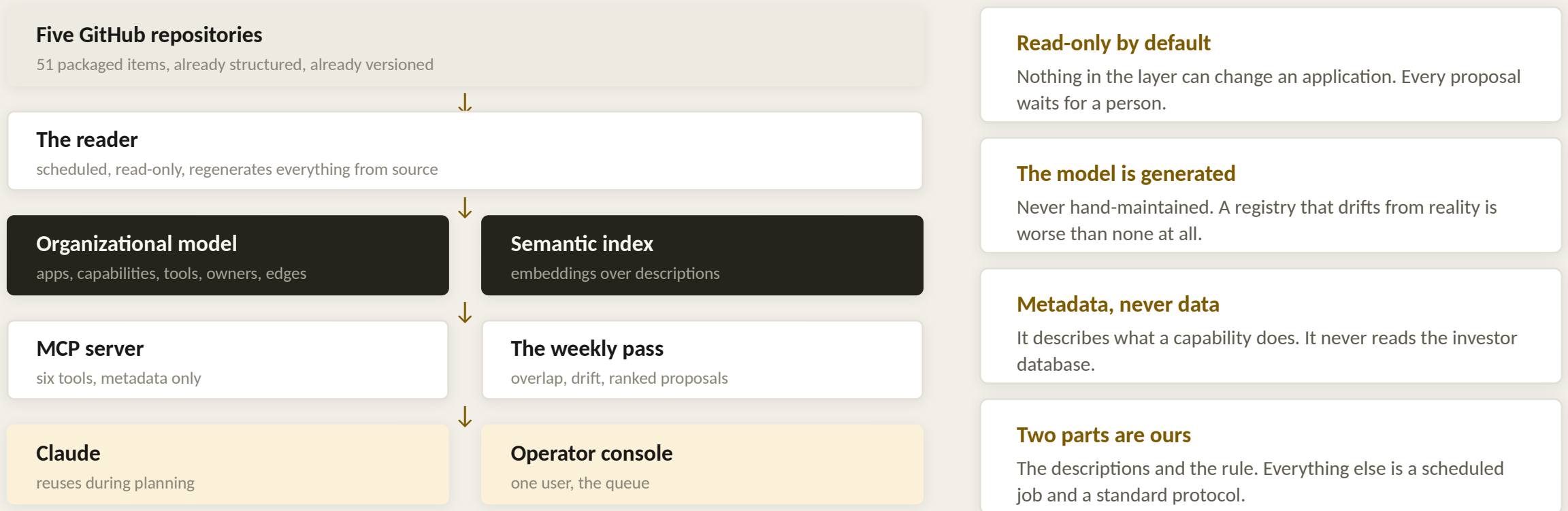
2 Is there a correctness or compliance stake?

3 Are they genuinely the same thing?

4 Can it be given a test?

All four, or it stays where it is. The default is not to centralize.

Six parts. Only two of them are ours.



Almost none of this is new technology.

Source of truth	GitHub, unchanged	existing
Scheduling	GitHub Actions on a cron	existing
The model store	Flat files in a repository, or Postgres if it outgrows that	ours
Capability descriptions	Written to be read by a model, generated and reviewed	ours
Semantic search	Off-the-shelf embeddings plus a vector index	bought
MCP server	TypeScript MCP SDK, deployed on Vercel or Railway	thin
Evaluations	Their existing eval suite, wired into MERGE-GATE	existing
Console	A static page reading the model	thin
Observability	OpenTelemetry into whichever backend they pick	bought
Identity and secrets	Whatever the IT and security work lands on	external

Two rows are genuinely ours, and one of them is prose rather than code. Everything else is a cron job, a standard protocol, or something Capital Factory already runs.

This is not a Capital Factory problem.

Any organization that let capable people build with AI for eighteen months has this exact estate.

Dozens of applications

nobody has counted recently

Overlapping capabilities

discovered by accident

No registry

and almost no evaluations

Ownership in people's heads

and in individual repositories

Capital Factory is unusual only in having measured it.

An industry already sells the catalog we started with.

Developer portals

Port
Backstage
Cortex
OpsLevel

Applications, owners, deployments,
operational health

Agent management

ServiceNow AI Control Tower
Microsoft Agent 365
IBM watsonx Orchestrate
Kore.ai · Airia

Inventory, govern, observe, sometimes
orchestrate

AI supply chain and identity

IFrog AI Catalog
Okta for AI Agents
Drata

Skill registries, agent identity, compliance
evidence

Every one of them governs what already exists. Discover, inventory, assign owners, monitor, report. All of it after something is built.

Nobody is in the room when somebody decides to build.

What the market sells

- Inventory and governance
- Observability and cost
- Compliance evidence and identity
- All of it after the fact
- Priced for organizations a hundred times this size

What is missing

Prevention at plan time.

Capability descriptions written to be read by a model, reachable from inside the tool the builder already has open, answering the question before a line is written.

Because the missing piece is small, it does not have to compete with any of them. It can sit on top of whichever one a company already owns.

Build the small missing part. Buy or skip the rest.

DO NOT BUILD

- An agent catalog
- A governance engine
- An identity layer
- An observability stack
- A generic creation platform

These exist and are better funded than anything we would write.

DO BUILD

- Capability descriptions written for a model to read
- The rule for what gets shared
- The plan-time hook that puts them in front of Claude

Three things. None of them is a platform.

Keep the machinery thin, standard and replaceable. Make the memory durable and compounding.

If a first-party capability registry ships tomorrow, the content ports and only our plumbing is thrown away. That is a good day, not a wasted quarter.

Smaller than you would expect.

A weekly pass that reads the estate and produces a ranked, reasoned list of what is overlapping and what to do about it.

No shared core yet. Nothing changed in any application.

It cannot break anything

Read-only, so it ships while eight people keep building.

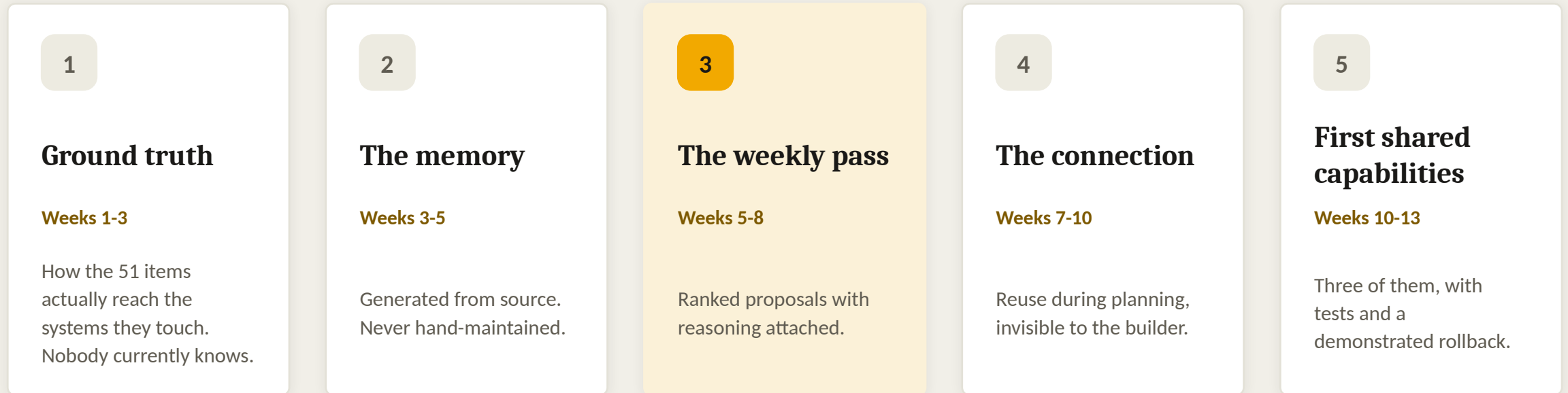
It is the instrument

You cannot pick the three capabilities worth sharing until you can measure which ones matter.

It stands alone

If the work stopped here, the list is still something anyone can act on.

Five stages, one acceptance test each.



stage 3 is the first shippable version

Stage 1 is the one nobody skips. Every estimate after it depends on its answer.

How you would know this was the wrong call.

Fewer than four capabilities pass the test

Then do not build a shared core at all. Keep the memory and the weekly pass, and stop there.

A first-party capability registry ships

One that reads the existing format. Adopt it, port the content, and throw our plumbing away.

Time from plan to working app goes up

Then the shared layer is wrong for this team, whatever it did to the duplication count.

These are written down on purpose, so the recommendation can be checked rather than trusted.

Every new application should make the next one easier to build.

That is the whole system. Everything else is machinery, and the machinery is meant to be replaceable.

Jorge Alejandro Diez

Architecture, system design, requirements and the build sequence are in the blueprint repository.